OS ORANGE-2

Name: Sharanya R

SRN : PES1UG24CS902

Section: A

Screenshot 1A

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$
```

Screenshot 2A

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ TREE_FILE=$(find .pes/objects -type f | head -1)
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ xxd "$TREE_FILE" | head -20
00000000: 626c 6f62 2031 3200 6865 6c6c 6f20 776f  blob 12.hello wo
00000010: 726c 640a                                rld.
```

Screenshot 3A

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ echo "Hello" > file1.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ echo "World" > file2.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes add file1.txt file2.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  Makefile
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h

sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ cat .pes/index
100755 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 1776684493 6 file1.txt
100755 b3d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12 1776684496 6 file2.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$
```

Screenshot 3B

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ cat .pes/index
100755 66224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 1776684493 6 file1.txt
100755 b3d731c8f5ea38a1118f9300e2dc257dee11461d53f4a1340e6afc4878ebec12 1776684496 6 file2.txt
```

Screenshot 4A

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ rm -rf .pes
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ echo "Hello" > hello.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes add hello.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 9007ab2da3c7... Initial commit
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ echo "World" >> hello.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes add hello.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes commit -m "Add world"
Committed: 23400fa51932... Add world
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ echo "Goodbye" > bye.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes add bye.txt
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes commit -m "Add farewell"
Committed: ab34b6351183... Add farewell
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ ./pes log
commit ab34b6351183a7cdb3f2227149e89a9b702c42e26f3f81e42e571af1c57a3894
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686648

    Add farewell

commit 23400fa5193229c3a5f5d0aaf8765a376d0651c0fe7d6466e6757105e28e6898
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686639

    Add world

commit 9007ab2da3c77d11bb1f7567f24a7b07507d802bf52036ef6b2d5677413fbbc3
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686632

    Initial commit
```

Screenshot 4B

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/23/400fa5193229c3a5f5d0aaf8765a376d0651c0fe7d6466e6757105e28e6898
.pes/objects/55/8c1061e21bccbeb23c527d8dcef5c8646cabfb6091aae3ae7ee3320c9ff7b7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/90/07ab2da3c77d11bb1f7567f24a7b07507d802bf52036ef6b2d5677413fbbc3
.pes/objects/ab/34b6351183a7cdb3f2227149e89a9b702c42e26f3f81e42e571af1c57a3894
.pes/objects/cc/e49eb2943f54e8135f291862bdf218064409ba05a7ee7d5bec00ddc22addfb
.pes/objects/dd/82340b5e403e2dc18bfca050e6699935849710b2b5d28ab8bbffbb72d32627
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ cat .pes/refs/heads/main
ab34b6351183a7cdb3f2227149e89a9b702c42e26f3f81e42e571af1c57a3894
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Full Integration Test

```
sharanya@DESKTOP-BPJ1TJ5:/mnt/c/Users/Bhavana/Downloads/os_orange/PES1UG24CS902-pes-vcs$ make test-integratio
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 8d0bba25dd10... Initial commit

Log after first commit:
commit 8d0bba25dd10ebfb3e23079868be6cf6c8ec50c74ac934f9aedd09ab0457b41a
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686699

    Initial commit


--- Second Commit ---
Committed: 67bb332b995c... Update file.txt

--- Third Commit ---
Committed: 54f23fedb4e7... Add farewell

--- Full History ---
commit 54f23fedb4e7bc6ff71cf5d8c2abaf94e60c3f9df04ab9eb3cfc447160b58a0e
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686699

    Add farewell

commit 67bb332b995c11085e8a472b507dda823c7c3cdccff56c5776a22cfffb97bf81
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686699

    Update file.txt

commit 8d0bba25dd10ebfb3e23079868be6cf6c8ec50c74ac934f9aedd09ab0457b41a
Author: Sharanya R  <PES1UG24CS902>
Date:   1776686699

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
54f23fedb4e7bc6ff71cf5d8c2abaf94e60c3f9df04ab9eb3cfc447160b58a0e

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b52536114 7faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/54/f23fedb4e7bc6ff71cf5d8c2abaf94e60c3f9df04ab9eb3cfc447160b58a0e
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/67/bb332b995c11085e8a472b507dda823c7c3cdccff56c5776a22cfffb97bf81
.pes/objects/8d/0bba25dd10ebfb3e23079868be6cf6c8ec50c74ac934f9aedd09ab0457b41a
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Question 5.1

- I) .pes/HEAD :
    - Currently, HEAD contains ref: refs/heads/main. After pes checkout feature, it must contain ref: refs/heads/feature. This is because the checkout must overwrite .pes/HEAD to point to new branch
- II) Working Directory
    - The files of working directory must be replaced with the files from the target commit's tree instead, and every tracked file must be restored to its content in that commit's snapshot.
- III) .pes/index
    - After checkout, the index must mirror the target commit's tree, and the file should be updated to match the checked-out commit.
- The current commit (HEAD) should be read to compute the difference between current tree and target tree. This is required to identify the files which need to be updated, deleted, or left unchanged

Implementation:

- Read .pes/refs/heads/<branch> to get the commit SHA
- object_read(target_commit_hash) to parse as commit
- Recursively parse the tree to get a list of (filename, blob SHA)
- For every leaf blob entry perform object_read(blob_hash) to read the raw bytes, and write this to file
- Delete the files which exist in current tree but not in the target
- Rewrite .pes/index with the target tree entries

Complex:

- Nested Directories may increase the complexity because the read is recursively performed
- Large trees require multiple file operations
- Deletion of files that don't exist in the target branch may happen, and this requires computing symmetric difference between current and target trees
- Before changing a single file, it should be verified that no working-directory changes would be silently overwritten, otherwise user changes can get overwritten
- If checkout fails mid-way, then the working directory is left in a partially-switched state, which is inconsistent

Question 5.2

- To detect the dirty working directory, we can make use of IndexEntry struct, and track the index (last staged version), object store (committed versions) and working directory (files).
- Firstly, check if the working file is dirty relative to the index for each tracked file from index.
- Compare the blob using TreeEntry.hash for the current file with the target, by recursively going through the target tree. So the read file is hashed and then compared with the index SHA.
- Check for conflicts by comparing the following conditions :
    - working_file_hash != index_hash (implies that the working file has unstaged changes)
    - index_hash != target_hash (implies that the two branches differ for the file)
    - working_file_hash != target_hash (implies that it will be overwritten by the checkout)

  if the above conditions are satisfied, it results in a conflict, thus the checkout is refused.
- This works as the index tracks the latest safe version and avoids the conflict.
    - Files that exist in the target but not in the current index don't give rise to conflicts because they are new, untouched files.
    - Files that exist in the current index but not in the target tree must be checked for unstaged changes before deletion
        - If working_file_hash != index_hash, then the file has local modifications and checkout must refuse
        - If working_file_hash == index_hash, then it is safe to delete the file during checkout
    - When the user manually edits the file to match the target branch content, it is safely handled by working_file_hash != target_hash.

Question 5.3

- Regular heads are when the HEAD file contains ref: refs/heads/main, after which the commit takes place, whereas detached head contains a bare hash where SHA is committed directly.
- In this state, if commit is performed,
    - target_path becomes .pes/HEAD itself
    - The commit is created and stored correctly in .pes/objects/ , thus the parent itself is the current commit

    - HEAD moves forward to point to the new commit
    - But no branch file points to the commit performed, thus files under .pes/refs/heads/ are not updated
    - When another branch has checkout, then the current commit cannot be reached. The commits exist in the object store but no branch points to them.
- In order to recover the commits,
    - The current head (*cat .pes/HEAD*) can be noted before the switch
    - A branch can be created immediately using *pes branch recover* which points to the hash of the current HEAD branch
    - This ensures that the recovery branch is permanently anchored to the commits, and they cannot be collected by the gc.
    - After the switch, the hash can be found by either using reflog or scanning through all the present objects in .pes/objects.

Question 6.1

- object_write() creates a file which is never deleted. Unreachable objects are accumulated.
- We can use the Mark-and-Sweep Algorithm
    - Data structure – Hash set
        - It has O(1) complexity
        - It prevents revisiting previously visited nodes
        - It prevents infinite re-traversal as a DAG with shared objects would
    - In the mark phase, DFS/BFS traversal can be done
        - the branch's commit hash is read and added to the reachable set, and the exploring starts from that commit to find the roots
        - the hash is recursively traversed, and termination occurs when the hash is present in reachable
        - If the commit points to a tree, then the tree is traversed to get the files. If it points to a parent commit, then the parent is traversed to get the history.
        - In trees, all the hash entires are traversed to visit every file and subfolder. The blobs are the leaves, hence it does not proceed further
    - In the sweep phase, cleanup is performed
        - All files in .pes/objects, and the subdirectories are traversed
        - The hash is reconstructed to get the objectId

        - Check for presence of hash in the reachable set, which indicates that it is still being used. If it's not in the set then it is garbage, which are deleted
        - Empty directories are cleaned
    - For 100,000 commits and 50 branches, assuming each commit has 1 tree, 10 files per commit are touched
        - Commits = 100000
        - Trees = 100000
        - Blobs = 1000000
            - Therefore, at worst case there are 1.2 million objects which are created, ignoring the shared trees and blobs, which will decrease the number of visited objects.
            - Hence, the complexity is O(n) where n is the number of unique objects visited.

Question 6.2

- Garbage collector deletes objects, while commit creates them, thus leading to a conflict when they are simultaneously operating.
- The race condition exists here:
    - In the commit thread, tree is created, object is written, hash is computed for that blob, which is in the store. The tree points to the blob, and the tree object is written to the tmp file
    - In the GC thread, the ./pes/objects is scanned, where the blob previously commited is found, but when the branch refs are traversed, the blob is not found because the blob is unreachable as the new commit has not been linked to the branch. Thus, the GC calls unlink to delete the blob.
    - In the commit thread, the tree is renamed, and the branch is updated to point to the deleted commit.
    - After this, if the object is attempted to be read, then the read fails because of the corrupted repository.
    - This might cause missing objects, corrupted history or crashes during checkout.
- This is avoided by Git using:
    - Reachability protection to keep recent objects safe
    - Reflog protection which temporarily protects unreferenced objects
    - Locking using .lock files to prevent concurrent modification in order to prevent two GC processes from running simultaneously
    - Grace period where objects newer than a certain age are never deleted, and only if they are unreachable for a long time are they deleted